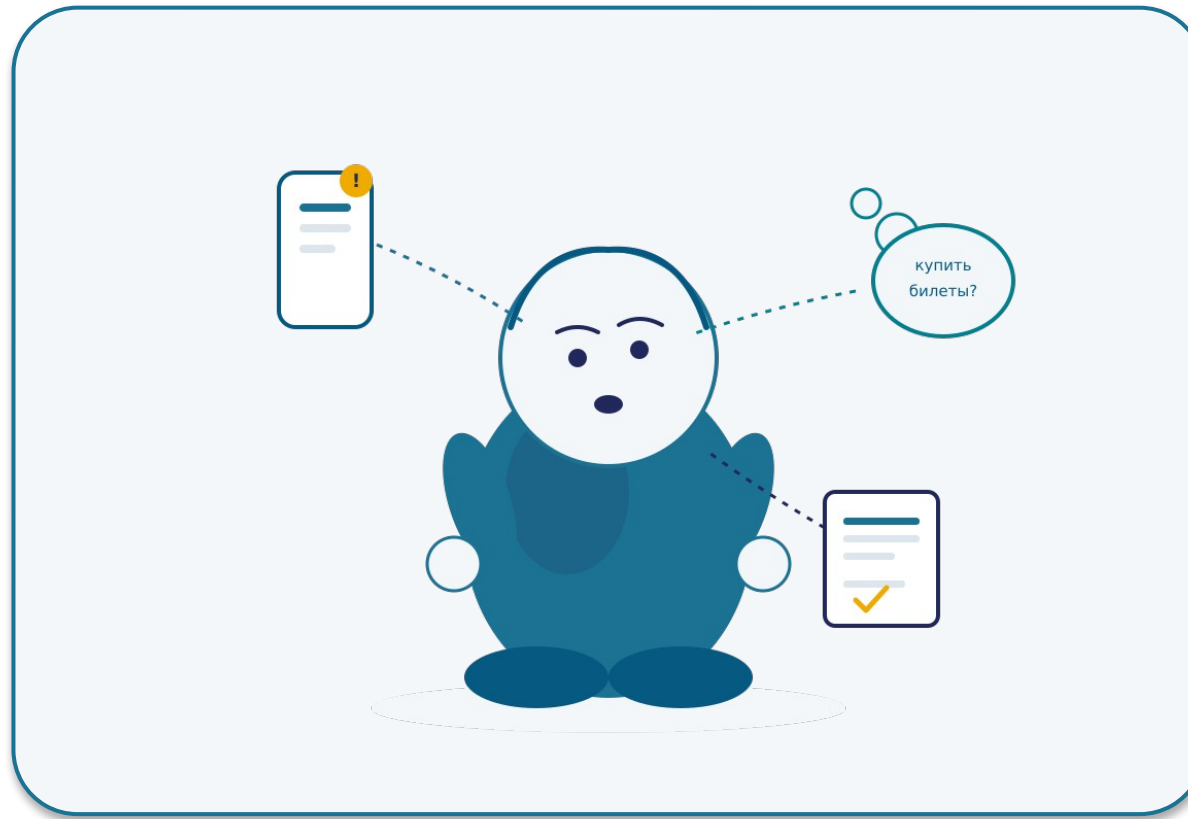


Пока вы читаете это предложение — на что вы НЕ обращаете внимания?

Ваш мозг каждую секунду выбирает, что важно, а что — фон.



Механизм, который сегодня разберём внутри модели ИИ, называется точно так же — «внимание». Он работает похоже — но не так, как ваш.

Как работают современные большие модели

4 этапа inference: токенизация · эмбеддинг · внимание · сэмплинг



Карта лекции — 6 разделов

0

Введение

*Что такое токен +
центральный вопрос*

1

Токенизация

*Как модель видит
ваш текст*

2

Эмбединги

*Пространство
смыслов*

3

Внимание

*Что важно
сейчас*

4

Сэмплинг

*От распределения
к токену*

5

Финал

*Закрытие,
мост к Л3*

Сегодня углубляем слой «модель» из четырёх слоёв Лекции 1

(из Лекции 1)

Приложение

Агент

Чат

МОДЕЛЬ — углубляем сегодня

Что мы знаем:

Модель = stateless inference. Вход — данные, выход — предсказание. Между вызовами памяти нет.

Что узнаем сегодня:

Что внутри inference. 4 этапа:

токенизация → эмбеддинг → внимание → сэмплинг

Главный вопрос лекции

«Что происходит внутри LLM между моим запросом и ответом — и какие из этих механизмов меняют, как я её использую?»

3 ответа — финал лекции

1

Почему промпт с ролью работает лучше пустого?

→ Раздел 3 — внимание

2

Почему AI плохо считает буквы?

→ Раздел 1 — токенизация

3

Почему один запрос даёт разные ответы?

→ Раздел 4 — сэмплинг

Раздел 1

Токенизация

Как модель видит ваш текст

0 Введение

1 Токены

2 Эмбединги

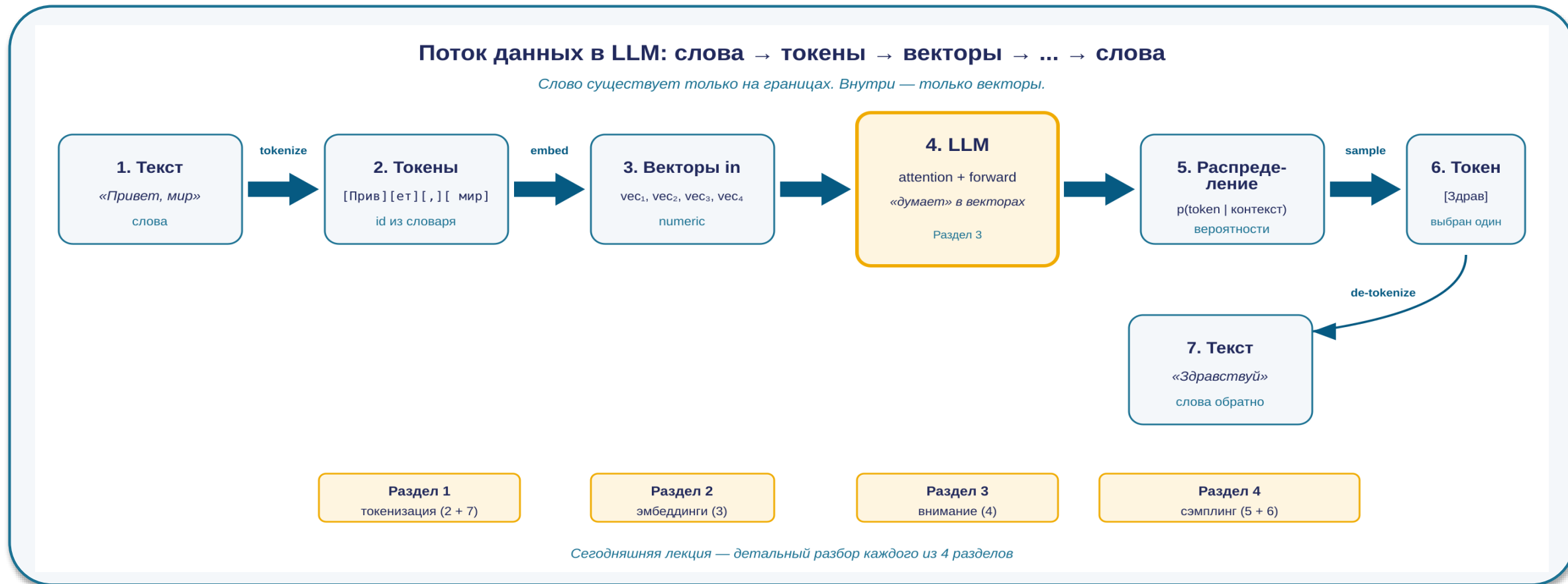
3 Внимание

4 Сэмплинг

5 Финал

Поток данных в LLM — туда и обратно

Каждый из этапов разберём отдельно. Сейчас — карта целиком.



Слово существует только на границах. Внутри модели — только векторы. Сегодня разбираем 4 этапа.

Токен — id из словаря модели. Не буква и не слово.

Статистически частая подпоследовательность

Пример 1.

cat

→

[cat]

1 токен / 1 id

Пример 2.

tokenization

→

[token]

[ization]

2 токена

Пример 3.

клубника

→

[к]

[луб]

[ника]

3 токена ·
o200k_base

В среднем: 1 токен ≈ 4 символа в EN ≈ 2 символа в RU

Для русского inference обходится примерно вдвое дороже — мы вернёмся к этому через 2 слайда.

BPE — компромисс между алфавитом и словарём

Словарь не из всех слов (как лемматизация) и не из всех букв (как *character-level*) — а из частых подпоследовательностей.

[1]

Словарь строится один раз перед обучением; в *inference* — *lookup* готовых правил.

Before (обучающий корпус)

- low
- lower
- newest
- widest



After (BPE-словарь)

- low
- er
- new
- est
- wid

BPE-словарь строится один раз до обучения. В *inference* — *lookup* готовых правил, не *runtime*-вычисление.

AI ошибается в «сколько r в strawberry» — слова не из букв, а из 3 токенов

strawberry в o200k_base (GPT-4o)

10 букв — то, что видит человек:

s t r a w b e r r y



3 токена — то, что видит модель:

st raw berry

Модель видит 3 единицы — не 10 букв

Подсчёт символов

«Сколько r в strawberry?» — ломается систематически, неочевидно для пользователя.

Опечатки

methodlogy → другие токены, чем methodology. Маленькая опечатка → большой сдвиг в ответе.

Регистр и пробелы

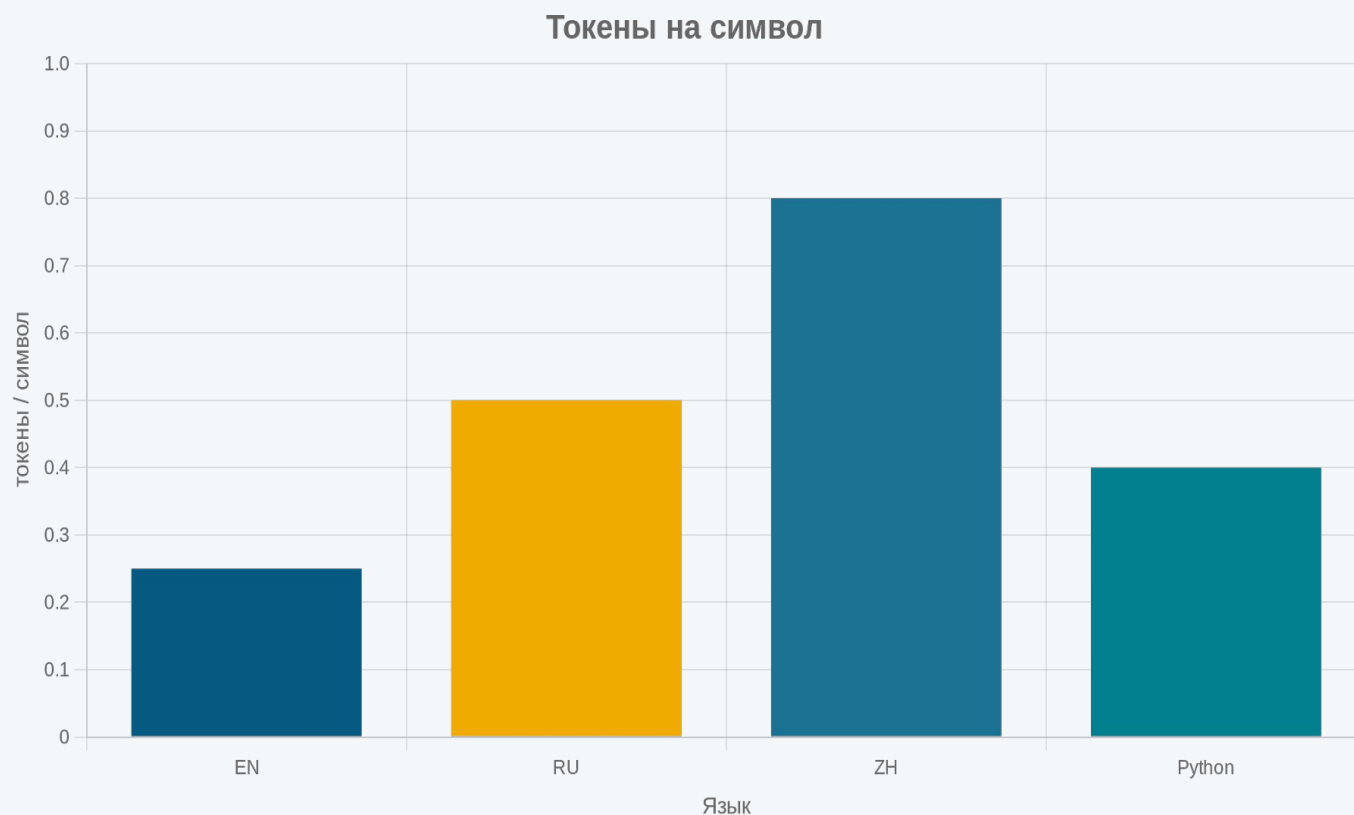
cat, ` cat`, Cat, CAT — разные токены, разные id.

Буквы: внешний инструмент (Python, regex) или посимвольный запрос. Топ-модели 2026 сами вызывают код вместо одного прохода.

Числа: тоже режутся непредсказуемо (1234 → 12+34, но 55688 → 556+88) → GPT-4: 59% точности на 3-знач. умножении, 4% на 4-знач., 0% на 5-знач. без калькулятора.^[1]

Один и тот же текст по-русски стоит в 2× дороже, чем по-английски

Токены на символ — следствие распределения языков в обучающем корпусе



Ориентир токены/символ

Английский	0.25
Русский (gold)	0.50
Китайский	0.80
Python-код	0.40

API-стоимость RU $\approx$ 2× от EN. Для batch — переводить в EN, если допустимо.

Раздел 2

Эмбе́ддинги

Пространство смыслов

0 Введение

1 Токены

2 Эмбе́ддинги

3 Внимание

4 Сэмплинг

5 Финал

Каждому токenu сопоставлен вектор — выучен на тренировке, фиксирован

Эмбеddинг: id → выученный вектор

[КОТ]

id = 47284

lookup в
embedding table →

[0.21, -0.45, 0.88, ..., 0.13]

несколько сотен — несколько тысяч измерений
выучен на тренировке, в inference фиксирован

Геометрическая близость = семантическая близость

Размерности (ориентир)^[1]

text-embedding-3-small

1536 dim

text-embedding-3-large

3072 dim

Внутренний эмбеddинг flagship LLM

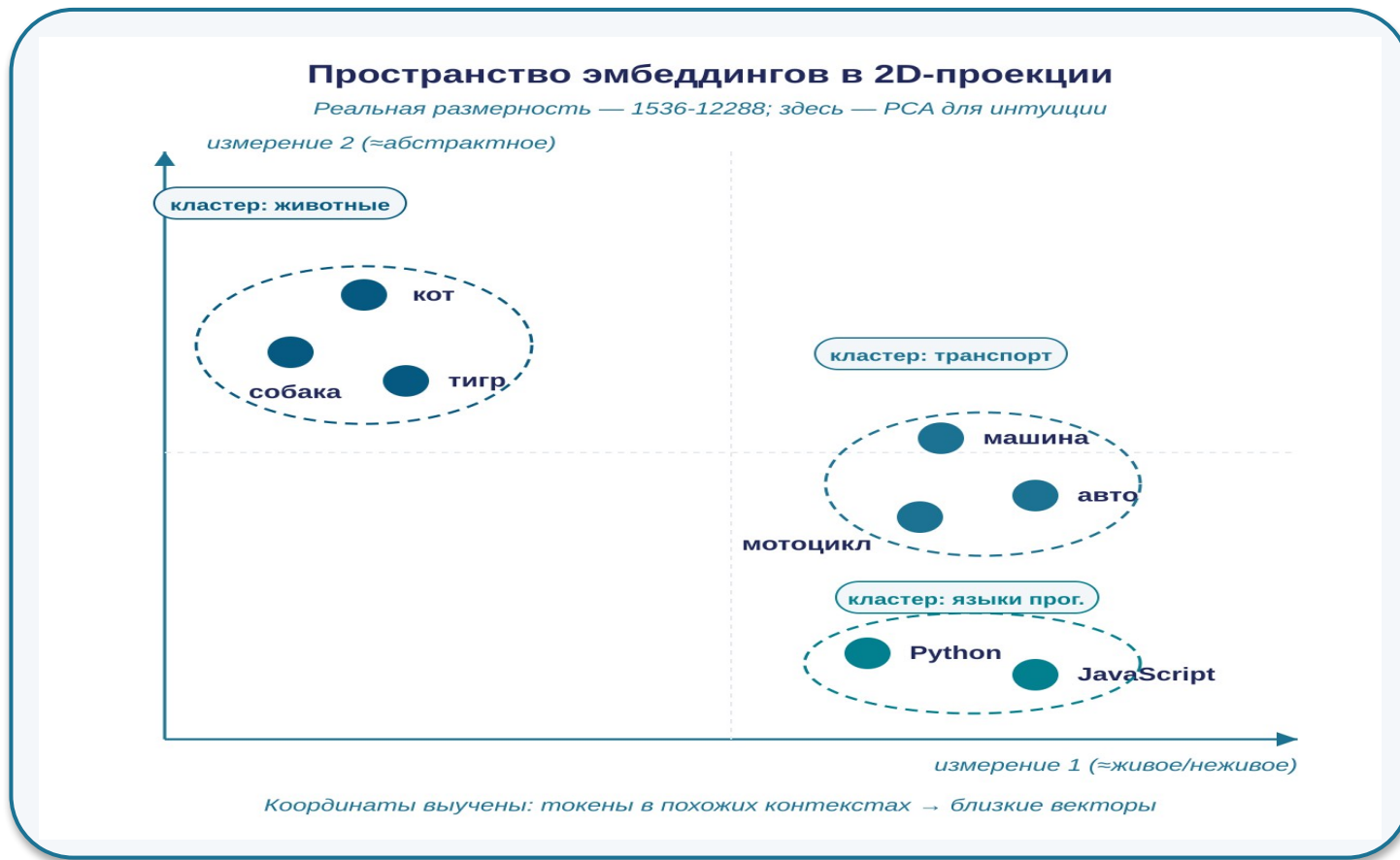
тысячи dim

Геометрическая близость векторов = семантическая близость токенов.^[2]

«Кот» близко к «собаке» — выучилось из контекстов корпуса.

Пространство эмбедингов — где живут векторы

Перед тем как сравнивать векторы — посмотрим, в каком пространстве они находятся.



1 Размерность

OpenAI text-embedding-3-small: 1536 dim.
text-embedding-3-large: 3072 dim.
Flagship LLM: тысячи.

2 Обучение

Координаты не задаются вручную. Модель учится:
токены в похожих контекстах → близкие векторы.

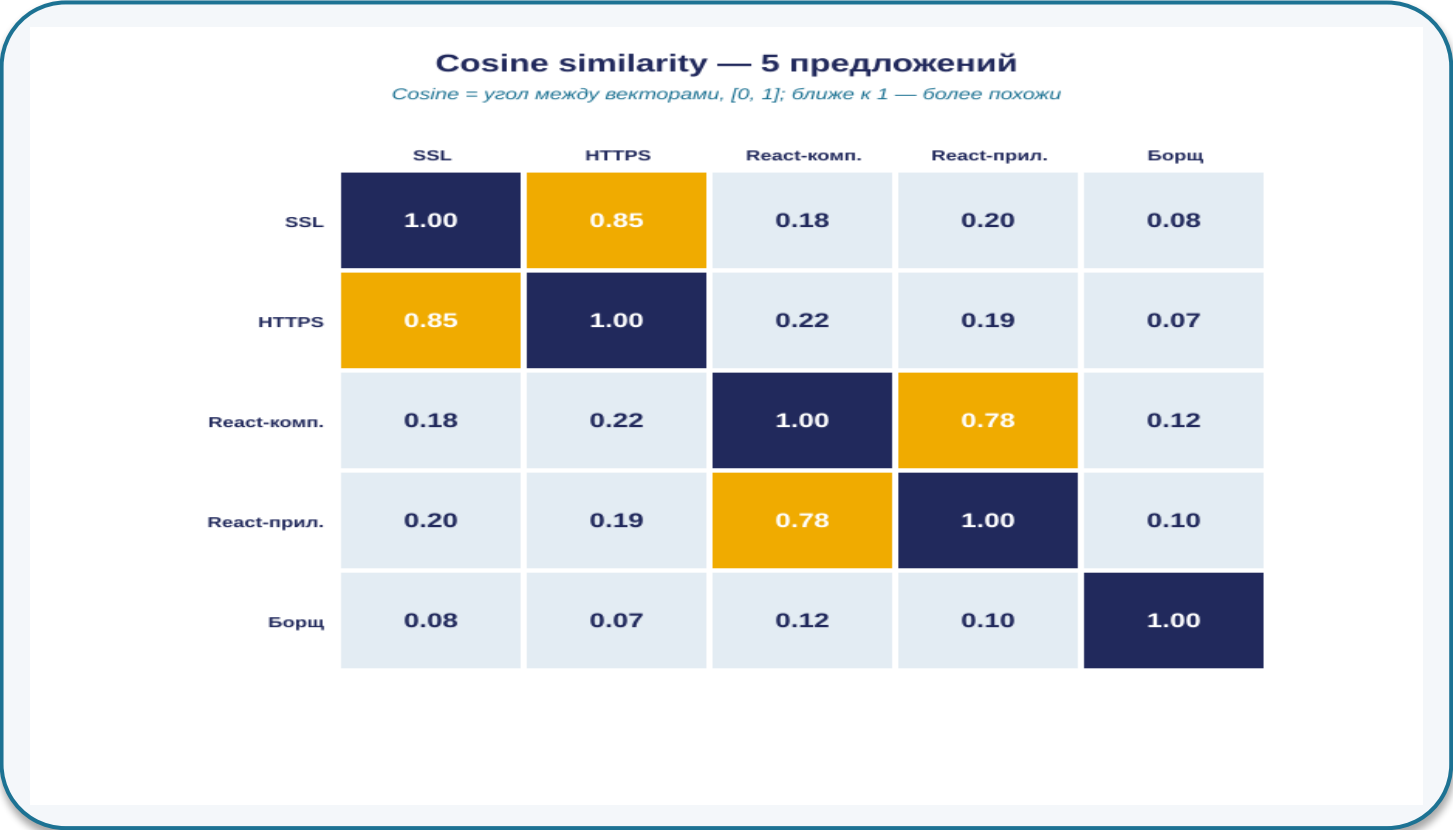
3 Проекция

Чтобы увидеть пространство — PCA или t-SNE
снижают размерность до 2D/3D.
Часть структуры теряется.

Семантическая близость = геометрическая близость векторов. На следующем слайде — мера.

Близость в пространстве эмбедингов = семантическая близость

В 2026 это работает на уровне предложений, не только слов

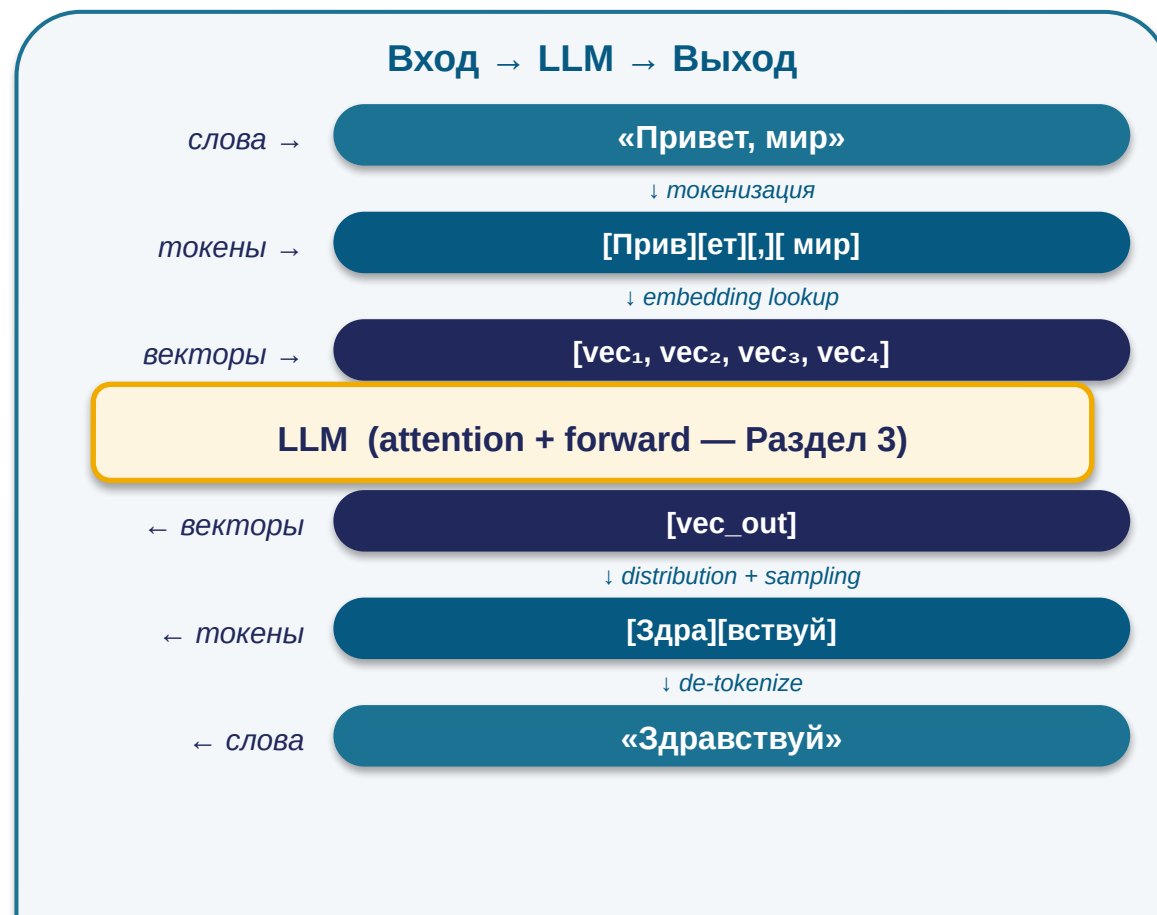


Cosine similarity — мера угла между векторами; диапазон [-1, 1], ближе к 1 — более похожи. Числа illustrative.^[1]

[1] воспроизводимо: [all-MiniLM-L6-v2 / text-embedding-3-small](#)

Эмбединги — фундамент понимания LLM

Внутри модель работает только с векторами; слова — только на входе и на выходе.



Что это даёт «понимание»



Перепарафразирования

«Как настроить SSL» и «Установка HTTPS» → близкие векторы
→ одинаковый ответ.



Синонимы

«авто» и «машина» → близкие векторы → одна и та же реакция.



Cross-lang

«клубника» и strawberry → близкие векторы → корректный ответ на любом языке.

Семантическая близость на уровне предложений — основа того, что LLM «понимает» переформулировки.^[1]

Раздел 3

Механизм внимания

Как модель решает, что важно сейчас

0 Введение

1 Токены

2 Эмбединги

3 Внимание

4 Сэмплинг

5 Финал

Внимание — это сверка каждого токена со всеми остальными

Каждый токен «смотрит» на все остальные одновременно. На каждом шаге — $N \times N$ связей.

Матрица внимания 7×7 (single-head, упрощение)

«Кот съел мышь, потому что она была голодна»

	Кот	съел	мышь	потому что	она	была	голодна
Кот	1.0	0.3	0.2	0.1	0.1	0.1	0.0
съел	0.4	1.0	0.5	0.1	0.1	0.1	0.1
мышь	0.2	0.4	1.0	0.1	0.1	0.0	0.1
потому что	0.1	0.2	0.2	1.0	0.3	0.2	0.2
она	0.1	0.1	0.7	0.2	1.0	0.3	0.4
была	0.1	0.2	0.2	0.3	0.4	1.0	0.5
голодна	0.1	0.2	0.3	0.3	0.4	0.5	1.0

«она» смотрит на «мышь»
вес = 0.7 (gold cell)

$N \times N$ связей; для 100k
— ≈ 10 млрд чисел

1 Размерность

$N \times N$, где N — длина контекста.
Контекст 100k → матрица из ~10 млрд чисел.
→ квадратичная стоимость внимания.

2 На каждом шаге

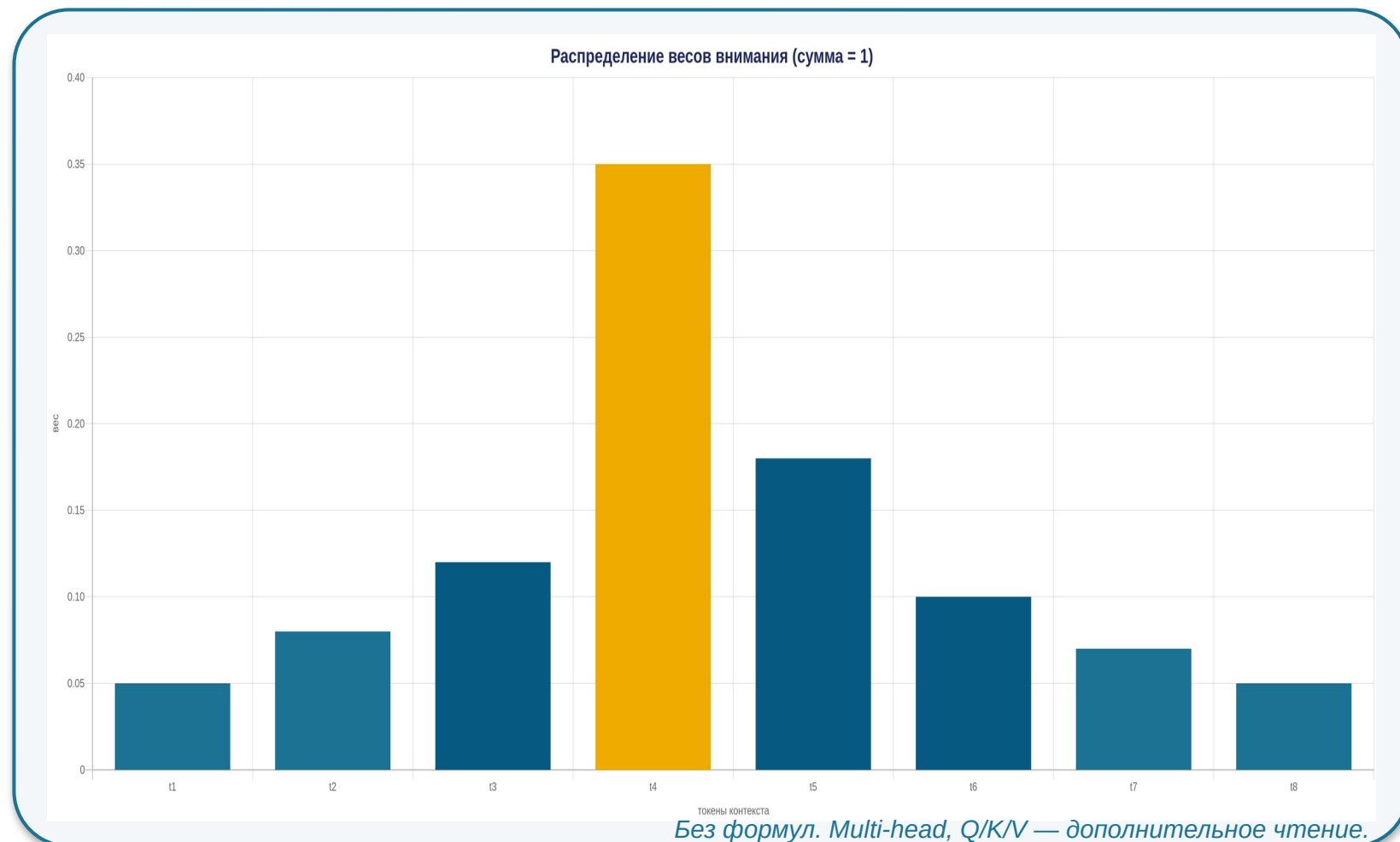
Матрица пересчитывается на каждом новом токене генерации (не один раз).

3 Multi-head

В одном слое — десятки таких матриц параллельно. Разные «головы» смотрят на грамматику, тему, дистанцию.

Внимание выдаёт распределение весов на все токены контекста (сумма = 1)

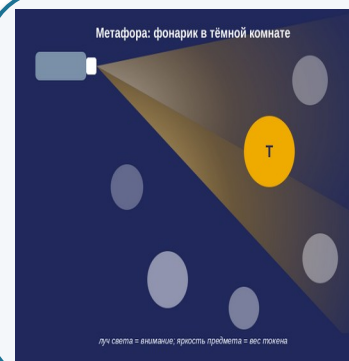
Какие токены сейчас важны для предсказания следующего^[1]



[1] [Vaswani et al. \(2017\) — Attention Is All You Need](#)

3 свойства

- 1 На вход — все токены контекста.
- 2 На выходе — распределение, $\Sigma = 1$.
- 3 Пересчитывается на каждом шаге.



Метафора:
фонарик в
тёмной
комнате —
модель
«подсвечивает»
одни
токены ярче
других.

Role-токены получают повышенный вес в весах внимания

Часть A — рабочий пример; часть B — эффект роли (1-е из 3 «почему»)

А. Разбор примера — куда смотрит «она»

«Кот съел мышь, потому что она была голодна»

она → мышь

толстая · главный вес

она → была

средняя

она → голодна

тонкая

Упрощение: реальная карта внимания — сотни связей. Модель смотрит статистически, не делает грамматический разбор.

Без роли

«Объясни асинхронность»

→ *обобщённый ответ*

(низкий вес role-токенов в весах внимания)

С ролью

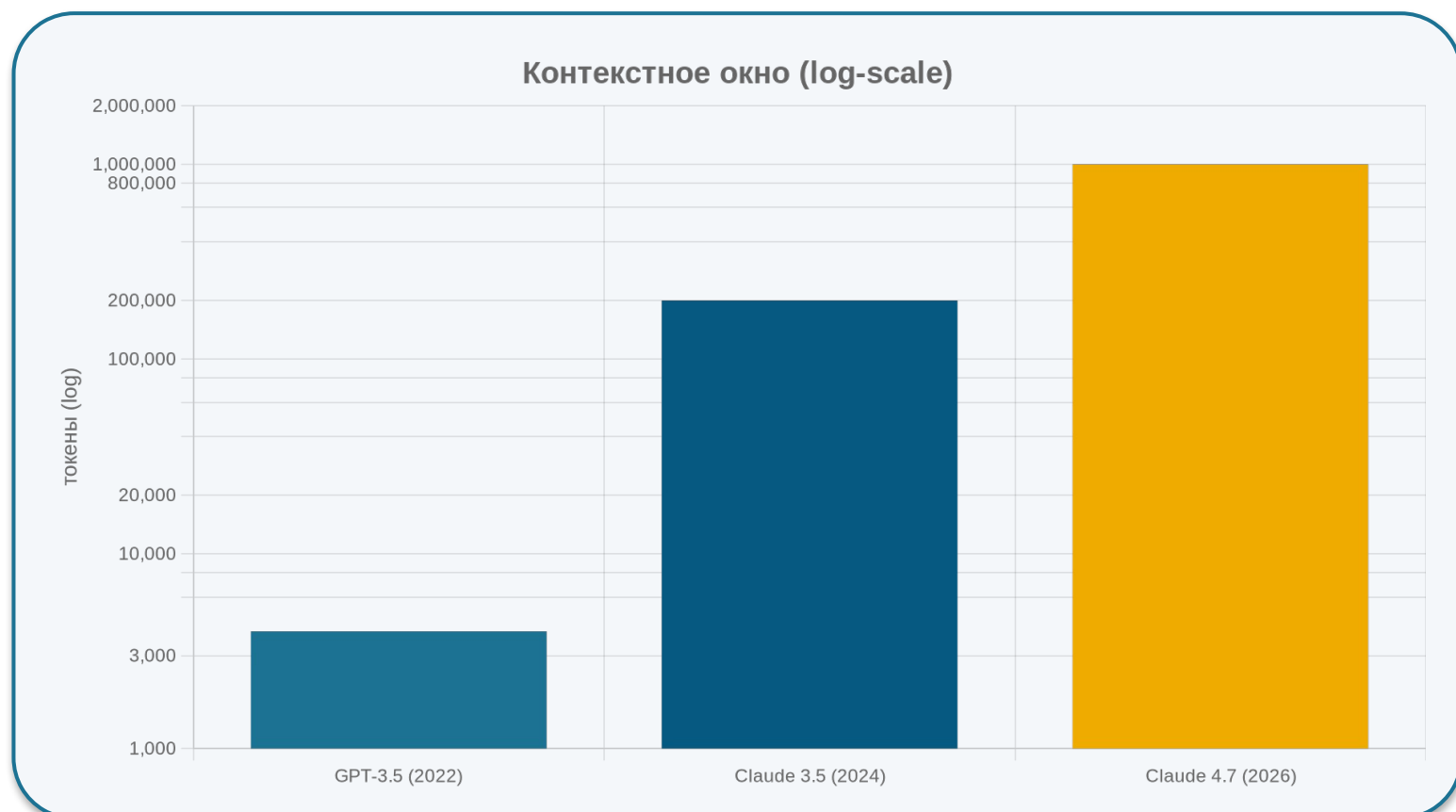
«Ты **эксперт по Python**. Объясни асинхронность **джуниору**.»

→ *role-токены подсвечены*

(высокий вес в весах внимания)

Контекстное окно — физический предел того, сколько модель видит одновременно

Эволюция контекстного окна + квадратичная стоимость внимания



Эволюция и стоимость

2022 → 2026: ×250 рост

Темп: ×10 / 1-2 года

Cost N²: 1M ≈ 16× от 100k

Архитектура: базовое внимание

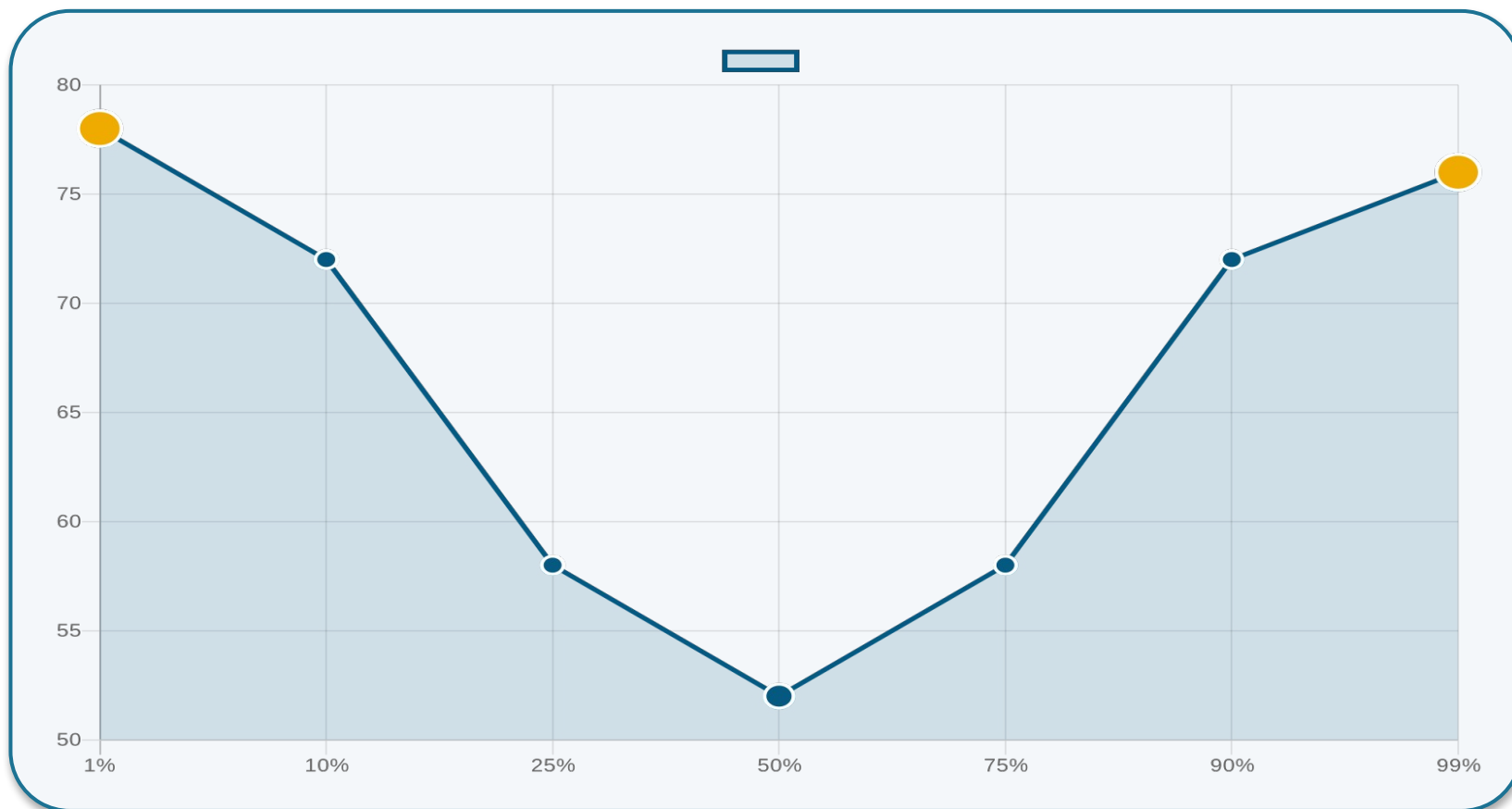
N²

Стоимость внимания растёт квадратично от длины. 1M ≈ 16× дороже 100k — production-pricing с batching; чистая N²-теория дала бы 100×.^[1]

[1] Anthropic — контекстное окно моделей Claude

Большое контекстное окно $\neq$ хорошее использование контекста

Lost-in-the-middle effect — модель забывает середину^[1]



Эксперимент

Факт вставлен в позицию X (начало / середина / конец) 100k-контекста. Модель отвечает на факт.

Результаты

Начало: ~75%

Середина: ~50%

Конец: ~75%

*Liu et al. 2023.
Lost in the Middle.*

Инженерный вывод: важное помещайте в начало или в конец промпта, не в середину.

[1] [Liu et al. \(2023\) — Lost in the Middle](#)

Раздел 4

Сэмплинг

От распределения к токену

0 Введение

1 Токены

2 Эмбединги

3 Внимание

4 Сэмплинг

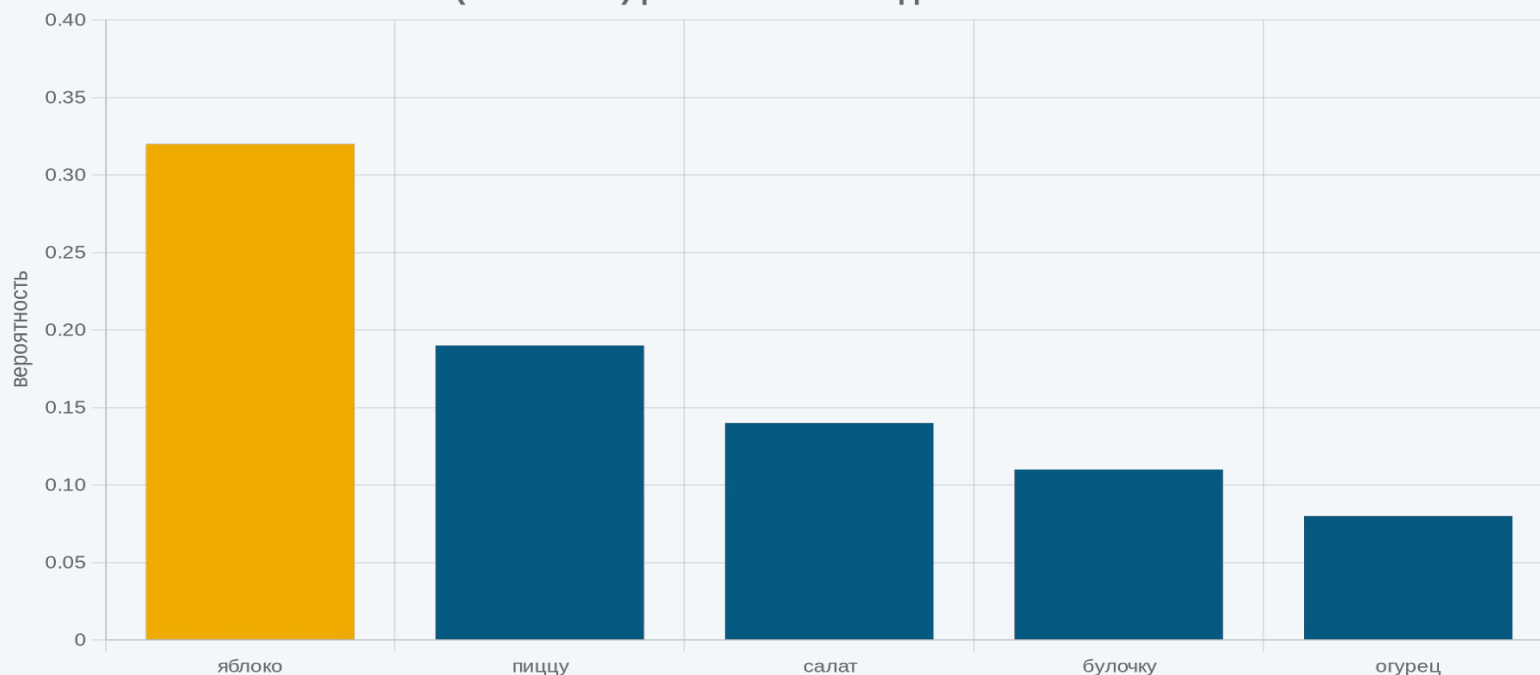
5 Финал

На каждом шаге модель выдаёт распределение вероятностей на все токены — выбирает один

Контекст

«Сегодня я съел ...»

P(next token) | контекст: «Сегодня я съел...»



Топ-5 кандидатов

яблоко	0.32
пиццу	0.19
салат	0.14
булочку	0.11
огурец	0.08

... остальные ~200k токенов:
каждый < 0.05

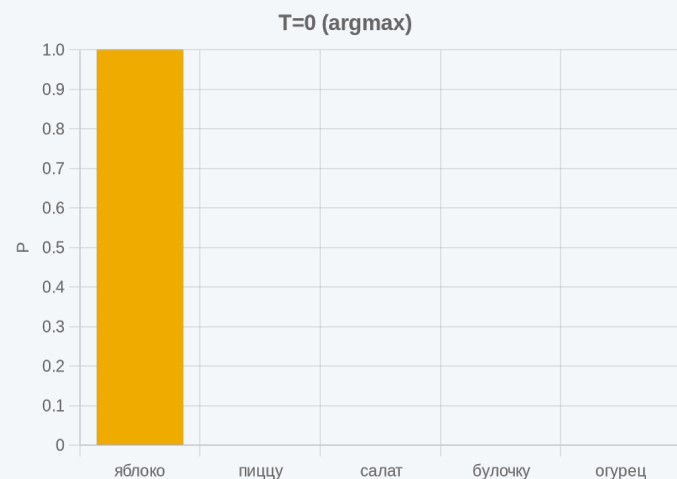
$$\Sigma = 1$$

Сэмплинг = правило, по которому из распределения выбирается один токен. Дальше — температура.

Температура: насколько острым будет выбор

$T = 0$ (argmax) · $T = 1.0$ (стандарт) · $T = 2.0$ (хаос)

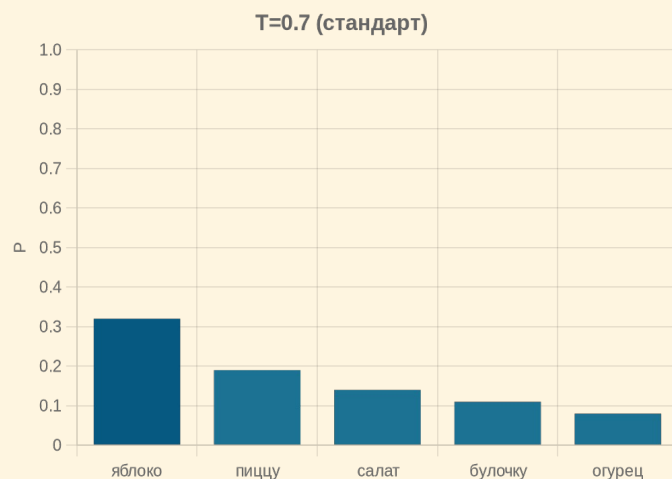
$T = 0$ · argmax



Детерминированный
выбор — яблоко.

10 запусков → одинаково.

$T = 1.0$ · стандарт

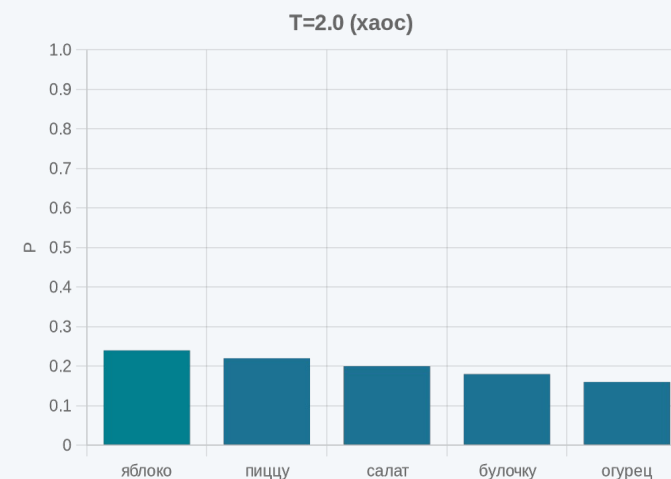


Сэмплирование
пропорционально P.

Естественная вариативность.

($T = 0.7$ — стандартный выбор для
чата)

$T = 2.0$ · хаос



Распределение сглажено;
часто выбираются

неожиданные варианты.

Альтернативные ручки: top-p (nucleus) — отрезает редкие токены по Σ ; top-k — по числу кандидатов. Достаточно T для start.^[1]

[1] [Holtzman et al. \(2019\) — nucleus sampling \(top-p\)](#)

4 ручки API под задачу: temperature, top_p, max_tokens, system prompt

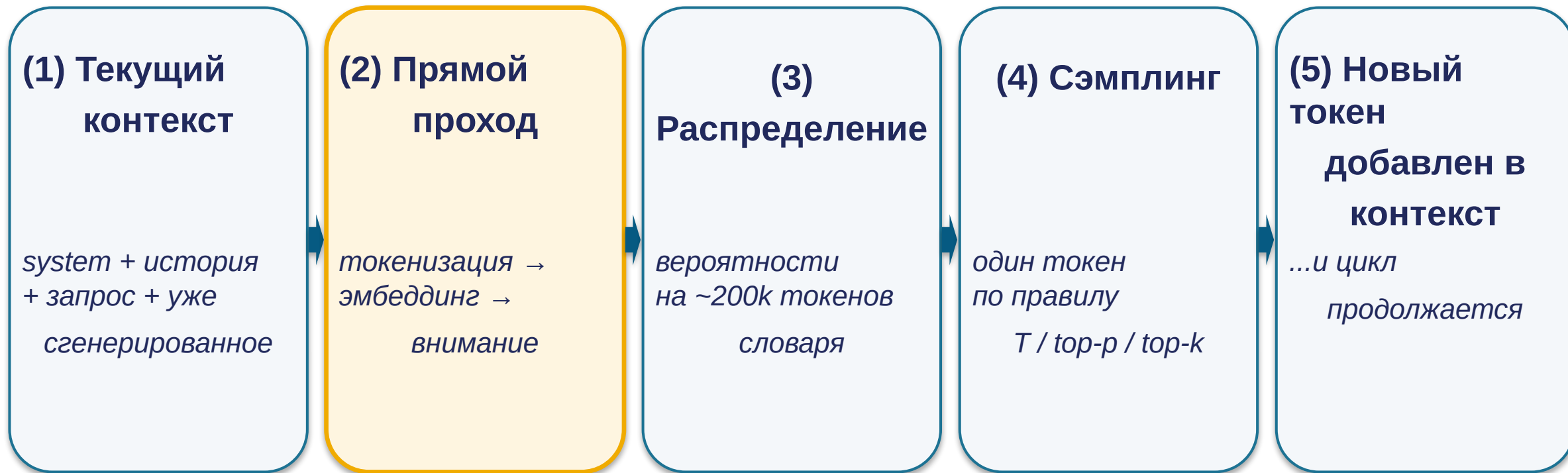
Подобрать параметры под сценарий обоснованно

Сценарий	temperature	top_p	max_tokens	system_prompt
Классификация / точное извлечение	0	—	50–200	Минимальный, со схемой выхода
Кодогенерация	0.2–0.3	0.9	1000+	Роль + контекст репозитория
Чат-объяснение пользователю	0.7	0.9	500–1000	Роль + описание аудитории
Творческое письмо	0.9–1.2	0.95	2000+	Роль + описание стиля

T = 0 практически детерминирует выбор; в production возможна микро-вариативность из-за batching — для большинства задач игнорируема.

Цикл: предсказали токен → добавили в контекст → предсказываем следующий

Авторегрессионная генерация — длинный ответ из stateless-вызовов



↪ возврат к шагу (1) — пока не дойдём до токена «конец ответа» ИЛИ до max_tokens

Inference одинаков локально и в облаке — но размер модели определяет качество

Архитектурно — тот же конвейер. Различия — в размере и среде.

Local (Ollama, llama.cpp, LM Studio)

Размер: 1–13B параметров

- Qwen 2.5 1.5B · Llama 3.2 1B
- Llama 3.1 8B · Mistral 7B^[1]

● Приватность	<i>запросы не уходят провайдеру</i>
● Скорость	<i>медленнее на consumer hardware</i>
● Контекст	<i>ограниченное окно</i>
● Цена	<i>0 за токен (своё железо)</i>

Cloud (OpenAI, Anthropic, Yandex, Сбер)

Размер: 200B+ параметров

- GPT-5, Claude 4.7
- YandexGPT, GigaChat
- Gemini

● Качество	<i>лучше на сложных задачах</i>
● Задержка	<i>200–500 мс</i>
● Контекст	<i>до 1M токенов</i>
● Цена	<i>оплата за токены, RU ≈ 2× EN</i>

[1] [Mistral AI — Mistral 7B](#)

Раздел 5

Финал

Заккрытие 3 «почему» + мост к Лекции 3

0 Введение

1 Токены

2 Эмбединги

3 Внимание

4 Сэмплинг

5 Финал

4 этапа inference сложились в конвейер

Тот же чёрный ящик из Лекции 1 — теперь распакован



Лекция 1 называла этот конвейер «inference моделью» — чёрным ящиком. Теперь он перестал быть чёрным.

Ответы на вопросы из начала лекции

1

Почему промпт с ролью работает лучше пустого?

На уровне внимания role-токены получают высокий вес — модель опирается на них при выборе следующих токенов.



2

Почему AI плохо считает буквы?

Токенизатор объединяет буквы в токены. strawberry — 3 токена, не 10 букв. Модель видит токены, не буквы.

01
10

3

Почему один и тот же запрос даёт разные ответы?

Сэмплинг — стохастический выбор из распределения при $T > 0$. Каждый запуск может выбрать разный токен.



Когда LLM — не правильный инструмент: дерево решений

Когда LLM — не правильный инструмент?



Фиксированные классы

Классификация на маленьком наборе категорий (5–20)?

→ Классический ML
лог. регрессия, XGBoost,
LightGBM, дообученный BERT



Интерпретируемость

Нужна интерпретируемость (финансы, медицина, страхование)?

→ Прозрачные методы
лог. регрессия + важность,
деревья решений, правила



Скорость отклика

Время отклика < 100 мс критично (антифрод, устройство пользователя)?

→ Специализированная
маленькая модель
(не LLM ≥ 200 мс)

Иначе — LLM подходит (chat, RAG, generation, многошаговое рассуждение)

Внимание статистически смотрит на токены — не понимает причинности

ИИ считает корреляции в данных, не строит каузальный граф



Человек

«X произошло, потому что Y»

Модель причинности — строит механизмы.

Опирается на физическую интуицию, доменные знания, знание механизмов мира. Для причинных выводов нужен именно этот опыт — не статистика.



ИИ (через внимание)

«X следует за Y в данных»

Статистическая корреляция, не причинность.^[1]

Замечает паттерн «X и Y часто соседствуют» в обучающих данных. Для причинных выводов — привлекайте эксперта предметной области или причинные методы.

^[1] [Pearl \(2018\) — The Book of Why](#)

Лекция 3: «Агенты, RAG, API — как AI выходит за пределы чата»

Все 4 концепции надстраиваются над одним проходом inference



RAG^[1]

Retrieval-Augmented Generation

близость эмбедингов + LLM → ответ из вашей базы



Инструменты / Вызов

функций

структурированный JSON

LLM генерирует вызов → выполняет внешняя система
→ результат возвращается



MCP^[2]

Model Context Protocol

Открытый стандарт подключения инструментов
(Anthropic, 2024)



Цикл агента^[3]

действуй → наблюдай → корректируй

Модель решает действие, видит результат,
корректирует план

Q&A

Спасибо